Pruebas en Lean y testigos finitos de los dos artículos sobre bicoloraciones de sistemas triples de Steiner

Juan Pablo Traverso Gianini
Investigador independiente, Santiago, Chile
jtraverso@gmail.com
ORCID: 0009-0003-6068-4096

Documento técnico de los artículos I y II.
Preprint 1.0 — 6 de noviembre de 2026.

Este documento identifica las fuentes de las pruebas de los dos manuscritos y precisa el alcance de su verificación. El teorema del espectro se aplica a todos los perfiles admisibles de orden suficientemente grande. La caracterización de tamaños de clase cubre todos los perfiles positivos y su demostración incluye un cálculo explícito con tablas finitas.

El resultado principal del artículo I es cdr_type_spectrum. La declaración auxiliar cdr_eventually aplica T al grafo completo (proposición 3.2). El artículo II demuestra existencia para todo orden. En la prueba escrita, los últimos casos finitos se deducen de CDR 1999; la prueba formal archivada verifica tablas explícitas.

1. Fuentes, módulo principal y alcance de la compilación

La referencia matemática es la revisión 7643193 de CDR [S1]. El identificador completo del commit es **76431935a1d61dbd7d512f32c941e37de38f6ba0**. Su registro de compilación da cuenta de 10 331 tareas de Lake completadas. Esta revisión permite reproducir el criterio 6/7 y la base finita registrados. Registro de compilación [S2] · Auditoría de axiomas [S3].

La revisión usa Lean 4.35.0-rc3 y Mathlib c55e6e786f49471c72fbddbec5415808896aec1e. La configuración de Lake localiza las dependencias mediante rutas locales. CDR usa los 999 módulos que corresponden al cierre de importaciones seleccionado de Paper III. El registro de procedencia distingue los hashes de las fuentes originales de los obtenidos tras incorporar las opciones de recursos del compilador. Al comparar con el archivo publicado de Paper III se observan también cambios en comentarios históricos y opciones de recursos insertadas. No se hallaron cambios en el código matemático de los 999 módulos. Para el conteo se usan otros 27 módulos con rutas de importación renombradas. Estas dependencias forman parte de la compilación de CDR. Paper III cuenta con su propio paquete publicado. Registro de procedencia · Paper III (DOI 10.5281/zenodo.23867836).

2. Cómo leer la prueba formal del espectro

Los nombres abreviados de esta sección llevan el prefijo CDRBicolour.Absorption y el subespacio de nombres que indica la fuente enlazada. El módulo raíz importa los resultados de la tabla. El informe calcula los axiomas de las declaraciones individuales.

Paso matemático	Fuente formal
Teorema 1, ambas implicaciones y criterio aritmético exacto	**Corners.Spectrum** (`CDRBicolour/Absorption/Corners/Spectrum.lean`), declaración **cdr_type_spectrum**
Balance en cada vértice, cociente y resto, obstrucción de grado cero	**Corners.Necessary** (`CDRBicolour/Absorption/Corners/Necessary.lean`)
Dos descomposiciones intercambiables y conteo exacto en el interior	**Trade** (`CDRBicolour/Absorption/Exact/Trade.lean`), **Exact.Final** (`CDRBicolour/Absorption/Exact/Final.lean`), **cdr_aac_exact**
Colocación inicial, corrección de la semilla y completación con tres tipos	**Corners.LowerT** (`CDRBicolour/Absorption/Corners/LowerT.lean`), **corner_lower**; **Seed3** (`CDRBicolour/Absorption/Corners/Seed3.lean`)
Extender un empaquetamiento o prescribir la intersección de dos sistemas	**Completion** (`CDRBicolour/Absorption/Completion.lean`), **cdr_completion**; **Refined.Completion** (`CDRBicolour/Absorption/Refined/Completion.lean`), **cdr_prescribed_intersection**
Separar conteos de Pasch y contar sistemas etiquetados	**PaschDiversity** (`CDRBicolour/Absorption/PaschDiversity.lean`), **cdr_many_pasch_counts**; **Wilson.Final** (`CDRBicolour/Absorption/Count/Wilson/Final.lean`), **realCount_log_asymptotic**

Los tres axiomas estándar del informe son la extensionalidad proposicional, la elección clásica y la regla de los cocientes, denominados en Lean propext, Classical.choice y Quot.sound. Son las únicas dependencias axiomáticas que el informe registra para el espectro y los demás resultados asintóticos de la tabla. El artículo desarrolla el conteo en un punto, el intercambio de una unidad, las aplicaciones de T y T+ y la unión de las pruebas de necesidad y suficiencia. Las pruebas extensas de los lemas de inversión, colocación, retículo y conteo se encuentran en las fuentes enlazadas.

3. Cómo leer la prueba para todo orden

La declaración final es **cdr_characterization**, en el espacio de nombres CDRBicolour. Supone positivos los tres tamaños y da la equivalencia entre realización y admisibilidad. Su demostración de suficiencia es independiente del uso de Paper III en el artículo sobre el espectro. **Caracterización** (`CDRBicolour/Characterization.lean`). La implicación necesaria, admissible_of_realises, usa las condiciones de la proposición 1.2 de Dinitz y Stinson. Las rutas que siguen corresponden al archivo ZIP común.

Paso matemático	Fuente formal
Producto, asignación de las clases y descenso en orden compuesto	**Product** (`CDRBicolour/Product.lean`); **Descent.L165** (`CDRBicolour/Descent/L165.lean`)
Coloración equitativa y emparejamientos perfectos finales	**Master.Equitable** (`CDRBicolour/Master/Equitable.lean`), **eqMaster_realises**
Diferencia cuatro y grafo restante en la familia de diferencia dos	**Master.L168** (`CDRBicolour/Master/L168.lean`); **L183.P2** (`CDRBicolour/L183/P2.lean`)
Emparejar los colores que faltan en un vértice y completar la familia estrecha	**L184.Assembly** (`CDRBicolour/L184/Assembly.lean`); **P2RouteClosed** (`CDRBicolour/P2RouteClosed.lean`)

Paso matemático	Fuente formal
Malla exhaustiva, testigos individuales e inducción	**Base.Cover** (`CDRBicolour/Base/Cover.lean`); **datos** (`CDRBicolour/Base/Data`); **Universal** (`CDRBicolour/Universal.lean`)

La cobertura de la malla se comprueba en el núcleo de Lean. En el informe de la revisión fijada, los tres enunciados para todo orden dependen de los tres axiomas estándar **y de 49 certificados de decisión por cómputo nativo**, uno por tabla. Para esta revisión decodificamos todas las tablas de forma independiente. Para cada par comprobamos el tercer punto, las dos consultas inversas y el patrón de colores. También cotejamos la lista de tablas con los perfiles que quedan después del barrido. Pasaron la comprobación **1 203 132 pares no ordenados**. En el primer artículo verificamos además que las dos filas impresas del intercambio coinciden con la fuente formal y cubren las mismas 27 aristas. Esta comprobación independiente con Python corrobora los datos. Los certificados nativos siguen formando parte de las dependencias de las declaraciones Lean.

4. Una instantánea común y dos informes de axiomas

Los artículos usan una sola revisión de las fuentes de CDR. Cada teorema importa las dependencias que requiere su prueba. La compilación registrada en esta edición completó 10 331 tareas de Lake. Las consultas archivadas de los módulos compilados en el equipo del autor coinciden con los conjuntos de axiomas registrados para las doce declaraciones finales. La auditoría de esta edición vuelve a comprobar esos registros y el archivo de fuentes; no ejecuta una nueva compilación de Lean. En una consulta aparte, la declaración de cobertura gridOK_eq resultó libre de axiomas. Las dos declaraciones importadas que usan 6/7 dependen únicamente de los tres axiomas estándar. El cálculo finito independiente comprobó 49 tablas, de orden máximo 487. El perfil de mayor orden es (154,156,177). La tabla de la sección 5 del artículo II reparte los 1551 perfiles en casos disjuntos. El perfil (696,702,745), de orden 2143, se resuelve por la rama equitativa y no requiere tabla.

Plantholt figura como PALOMAR-2026-11-04-000016 v2 (5 de noviembre), verificada por Comparator, nanoda y con-ron. La dependencia es la revisión **f8900944da7743b1facd771139c9ebdbfd8aaa28** de la rama pública principal del repositorio de Plantholt. Contiene el resultado de Chetwynd y Hilton de 1985 y su aplicación al criterio de Plantholt. En grafos regulares sobre $2n$ puntos, la hipótesis $12n \leq 7d$ equivale a $d \geq 6(2n)/7$. Fuentes de la dependencia [S4].

Para reproducir la auditoría hay que restaurar la revisión fijada de CDR y sus dependencias de ruta, incluida la adaptación de Vizing y los módulos renombrados para el conteo. Tras compilar el objetivo predeterminado de Lake se ejecutan checks/AxiomsFreeze.lean y checks/SharedInputs.lean en el entorno Lean del proyecto. El paquete de revisión incluye las salidas y un verificador independiente en Python de los datos finitos. Estas consultas utilizaron la compilación local ya terminada. El archivo incluye las dependencias con rutas relativas. Esta revisión comprueba el archivo y la compilación existente; una reconstrucción en otro equipo es un experimento aparte.

5. Cómo se enlazan las declaraciones de Lean con las pruebas

Existencia en orden grande. En Asymptotic.lean, cdr_eventually elige el umbral antes de fijar los tamaños de las tres clases. El grafo sobre el que se aplica el teorema es el completo con la partición prescrita. La prueba toma el mayor entre 400 y el umbral que devuelve main_theorem_T con parámetros $(1/32, 4, 6)$. Entrega a T las cuatro verificaciones cdr_H1,

cdr_H2, cdr_H3’ y cdr_H4. El último paso, realises_of_decomposition, convierte la descomposición en triángulos en el sistema triple buscado. Así se obtiene el resultado auxiliar de existencia de la proposición 3.2.

Las cuatro comprobaciones en el grafo completo se pueden leer por separado. **H1** (`CDRBicolour/Absorption/Host.lean`) · **H2** (`CDRBicolour/Absorption/Lattice.lean`) · **H3/H4** (`CDRBicolour/Absorption/Hinge.lean`) · **proposición 3.2** (`CDRBicolour/Absorption/Asymptotic.lean`).

Conteos exactos. Corners.Necessary parte de un sistema y cuenta los pares que pasan por un vértice. Corners.Spectrum reúne las dos condiciones resultantes en ExactNec. La prueba de suficiencia comprueba primero si se pueden colocar de antemano los bloques escasos en el orden original de las clases. Si se cumple la desigualdad requerida, Graphs construye los grados internos, Plant asigna los terceros puntos y Lower reúne los bloques elegidos. Residual calcula los números de vecinos que quedan; Seed3 y Lattice3 establecen las condiciones fraccionaria y entera en esas aristas. LowerT aplica T con los tres tipos mayoritarios. Si la desigualdad se cumple al intercambiar las clases, Symmetry devuelve la construcción al orden original. Si falla en ambos órdenes, large_counts_R sitúa los seis conteos en el intervalo que trata Exact.Final. Allí se aplica T+ y se eligen los intercambios reservados. Tanto la necesidad como la suficiencia se refieren al mismo entero t.

Los pasos intermedios del espectro exacto se encuentran en estas fuentes. **Sucesión de grados** (`CDRBicolour/Absorption/Corners/Graphs.lean`) · **Colocación** (`CDRBicolour/Absorption/Corners/Plant.lean`) · **Retículo entero** (`CDRBicolour/Absorption/Corners/Lattice3.lean`) · **Corrección de masa** (`CDRBicolour/Absorption/Exact/Final.lean`) · **División en casos** (`CDRBicolour/Absorption/Corners/Spectrum.lean`).

Todos los órdenes. Characterization da las dos implicaciones de la caracterización. Para la suficiencia, Universal ordena las clases y procede por inducción fuerte en su tamaño total. Descent.L165 combina sistemas menores en orden compuesto. Master.Equitable construye cinco tipos de bloque mediante coloraciones sucesivas de aristas. En la familia de diferencia dos, el empaquetamiento deja un grafo sobre la última clase. L183.P2 prueba que sus aristas se pueden colorear con el número requerido de colores y L184.Assembly incorpora los últimos pares. Base.Cover identifica los casos que quedan por debajo de 5059. Verificar un testigo demuestra una realización concreta. El teorema de cobertura y la inducción permiten usar esas realizaciones en el enunciado universal.

El registro de trabajo describe cómo se fueron obteniendo las construcciones. Para seguir las demostraciones de estos artículos, los enunciados y las importaciones de la revisión fijada son la referencia definitiva. Conviene leer las notas sobre los extremos junto con Corners.Spectrum, donde se reúnen los dos extremos y el intervalo interior. GATES · Construcciones en los extremos.

6. Cómo reproducir un testigo y leer la auditoría

Cada tabla guarda dos dígitos en base 64 para cada par no ordenado, en orden lexicográfico. Se decodifica el tercer punto, se comprueba que difiere de los dos dados y se buscan en la tabla los otros dos pares de la terna. Cada consulta debe devolver el punto que falta. Por último se comprueba que los tres puntos tienen exactamente dos colores. Para la menor tabla se recuperan $012, 034, 056, 136, 145, 235, 246$, los siete bloques de la figura 1 del artículo I. La comprobación de las 49 tablas recorre 1 203 132 pares y se puede repetir independientemente de Lean.

La primera orden verifica el paquete, el informe de Lean y los testigos desde el repositorio de publicación. Para repetir las consultas de declaraciones, extraiga el archivo común, restaure el entorno compilado y ejecute las dos últimas órdenes desde la raíz de ese proyecto. Estas

consultas leen objetos existentes. En una instalación que solo tenga las fuentes hay que obtener primero las dependencias fijadas y compilarlas.

```
python scripts/verify_repository.py
```

```
lake env lean checks/AxiomsFreeze.lean
```

```
lake env lean checks/SharedInputs.lean
```

7. Enunciados y alcance de la verificación

Se transcriben las definiciones y los enunciados de las fuentes congeladas. Los nombres son relativos a sus espacios de nombres. Las pruebas se encuentran en el archivo común.

```
def Realises (a b c : ℕ) : Prop :=
  ∃ (S : STS (Fin (a + b + c))) (col : Fin (a + b + c) → Fin 3),
    IsStrictBicolouring S col ∧
      classSize col 0 = a ∧ classSize col 1 = b ∧ classSize col 2 = c


def AdmissibleProfile (a b c : ℕ) : Prop :=
  0 < a ∧ 0 < b ∧ 0 < c ∧
  ((a + b + c) % 6 = 1 ∨ (a + b + c) % 6 = 3) ∧
  ((a : ℤ) ^ 2 + b ^ 2 + c ^ 2 - a * b - a * c - b * c = a + b + c) ∧
  ¬ (a = 2 ∧ b = 2) ∧ ¬ (a = 2 ∧ c = 2) ∧ ¬ (b = 2 ∧ c = 2)


def CDRConjecture : Prop :=
  ∀ a b c : ℕ, AdmissibleProfile a b c → Realises a b c


theorem cdr_conjecture : CDRConjecture


theorem cdr_characterization {a b c : ℕ} (ha : 0 < a) (hb : 0 < b) (hc : 0 < c) :
    Realises a b c ↔ AdmissibleProfile a b c


theorem cdr_characterization_bicolourable3 {a b c : ℕ} (ha : 0 < a) (hb : 0 < b) (hc :
0 < c) :
    CDR.Bicolourable3 (a + b + c) a b c ↔ AdmissibleProfile a b c


def LowerNec (a b c t : ℕ) : Prop :=
  ∃ d q r : Fin 3 → ℕ, (∀ X, r X < sz a b c X) ∧
    (∀ X, (d X : ℤ) = sz a b c X - 1 - sz a b c (X + 1) + 2 * q X) ∧
    (∀ X, sz a b c X * d X + 2 * r X = 2 * (sz a b c (X + 2) * q (X + 2) + r (X + 2)))
∧
    (∀ X, ¬ (d X = 0 ∧ (r X = 1 ∨ r X = 2))) ∧ c * q 2 + r 2 = t
```

```
def ExactNec (a b c t : ℕ) : Prop :=
  2 * t ≤ a * (a - 1) ∧ LowerNec a b c t ∧ LowerNec a c b (a * (a - 1) / 2 - t)


def AACSpec (a b c t : ℕ) : Prop :=
  ∃ S : STS (Fin (a + b + c)), IsStrictBicolouring S (col3 a b c) ∧ typeCount S (col3
a b c) 0 2 = t


theorem cdr_type_spectrum : ∃ n₀ : ℕ, ∀ a b c : ℕ, AdmissibleProfile a b c → n₀ ≤ a
+ b + c →
    ∀ t : ℕ, AACSpec a b c t ↔ ExactNec a b c t


theorem cdr_eventually :
    ∃ n₀ : ℕ, ∀ a b c : ℕ, AdmissibleProfile a b c → n₀ ≤ a + b + c → Realises a b
c
```

Cada comprobación nativa de una tabla confía en el compilador y en la reducción ejecutada. La versión usada de Lean imprime un axioma nativo generado distinto por tabla. El verificador independiente en Python no elimina esos axiomas. La adaptación de Vizing satisface el nombre de paquete exigido por las dependencias; las declaraciones reales pertenecen a las fuentes de LeanPool incluidas.

Agradecimientos

El autor agradece profundamente a su esposa María Paz y a sus hijos Lucas, Juan Cristóbal, Francisca, Raimundo y Benjamín su amor, paciencia y apoyo.

Evidencia del Teorema 6.2

El teorema de intersección prescrita tiene una prueba simbólica en Lean para todos los órdenes admisibles suficientemente grandes; no se certifica mediante una enumeración finita. Sus dos completaciones residuales usan clases disjuntas de residuos. El criterio elemental de intersección y la guía de dependencias formales aparecen en el Apéndice B del Paper I. El archivo de fuentes sin cambios incluye `checks/AxiomsIntersection.lean`. Tras restaurar las dependencias fijadas, se ejecutan `lake build CDRBicolour.Absorption.Refined.Completion` y `lake env lean checks/AxiomsIntersection.lean`. El script del repositorio `scripts/verify_intersection_evidence.py` comprueba únicamente la coherencia de las fuentes y los registros archivados; no ejecuta Lean. Esta corrección editorial no repitió la compilación de Lean.

Auditoría por declaración

Declaración (en CDRBicolour)	Axiomas registrados
`Absorption.Corners.cdr_type_spectrum`	3
`Absorption.cdr_eventually`	3
`Absorption.cdr_completion`	3
`Absorption.Refined.cdr_prescribed_intersection`	3
`Absorption.PaschSeed.cdr_many_pasch_counts`	3
`Absorption.PaschSeed.cdr_many_nonisomorphic`	3
`Absorption.cdr_aac_dense`	3
`Absorption.Exact.cdr_aac_exact`	3

Declaración (en CDRBicolour)	Axiomas registrados
`Absorption.Count.Wilson.realCount_log_asymptotic`	3
`cdr_conjecture`	3 + 49
`cdr_characterization`	3 + 49
`cdr_characterization_bicolourable3`	3 + 49

Referencias y herramientas computacionales

C. J. Colbourn, J. H. Dinitz y A. Rosa, *Bicoloring Steiner Triple Systems* (1999), Teorema 2.6. https://doi.org/10.37236/1457. J. P. Traverso Gianini, *Triangle decompositions III* (2026), v1.0. https://doi.org/10.5281/zenodo.23867836.

Claude, de Anthropic, y ChatGPT/Codex, de OpenAI, se utilizaron para explorar y comprobar argumentos y para preparar el manuscrito.

Certo y Jacobian se utilizaron para explorar casos finitos, comprobar identidades y evaluar construcciones candidatas. Estos cálculos orientaron el desarrollo de los argumentos; las comprobaciones finitas no se trataron como pruebas de enunciados universales.

El autor es responsable de los argumentos, las citas, el código y la exposición. Ningún sistema de IA figura como autor.

Registro de dependencias. CCJS: PALOMAR-2026-11-04-000009 v2. Dinitz–Stinson: PALOMAR-2026-11-04-000014. Rudašová–Soták: PALOMAR-2026-11-05-000001. Vizing: PALOMAR-2026-10-15-000004. Conteo: PALOMAR-2026-10-22-000017 v2. BKLS: PALOMAR-2026-11-06-000002 v1, registrado el 6 de noviembre de 2026 [S5]. Su revisión es 736c78e, la misma que usa la dependencia congelada.

Material suplementario

[S1] Revisión 7643193 de CDR. https://github.com/jtraverso/cdr-steiner-preprints/blob/v1.0/archives/CDR_shared_Lean_source_7643193.zip.

[S2] Registro de compilación. https://github.com/jtraverso/cdr-steiner-preprints/blob/v1.0/publication/audit/EXISTING_BUILD.json.

[S3] Auditoría de axiomas. https://github.com/jtraverso/cdr-steiner-preprints/blob/v1.0/publication/audit/live_axioms.txt.

[S4] Fuentes de la dependencia. https://github.com/jtraverso/plantholt-overfull-criterion/tree/f8900944da7743b1facd771139c9ebdbfd8aaa28.

[S5] Registro público de BKLS. https://palomar-registry.org/entry?id=PALOMAR-2026-11-06-000002&version=1.